

Arena

New Chat

Leaderboard

Search

Today

[Arena.ai][http://Ar...

Older

1. File Lock & OneDriv...

can you fix the followi...

Make it moving and g...

Make it moving and g...

any improvement tha...

**Prompt for Arena.ai...

We are running all 3 a...

Quant Auditor &...

AUDIT INSTRUCTI...

Engine_1 Operatio...

MISSION SPECIFI...

**# 🏹 PROP FIRM C...

Act as an expert CAT ...

⌵ Agent Mode

Arena.ai Prompt – Coinglass Trading Engine (Engine_1 / Multi-Strategy Optimizer)

Context & Project Overview

You are working on a live algorithmic trading system called Engine_1 — a multi-strategy, ML-enhanced, walk-forward optimized trading engine for crypto futures (BTCUSDT, ETHUSDT, SOLUSDT and 11 other pairs on Binance).

GitHub Repository: github.com/kbsingh1399/coinglass-trading (branch: master)

Please fetch and read the entire repository before proceeding. Focus on:

- [opt_s1_colab_standalone.py](#) — Strategy 1: Liquidation cascade entries
- [opt_s2_colab_standalone.py](#) — Strategy 2: CVD divergence
- [opt_s3_colab_standalone.py](#) — Strategy 3: Trend-pullback + RSI
- [Engine_1.py](#) — Live trading supervisor
- [supervisor.py](#) — TUI dashboard
- [ensemble_master/](#) — Ensemble combining S1-S6

What We Have Achieved

- 6-strategy modular engine (S1-S6), each independently optimized
- 20-window expanding walk-forward validation across 6 years of 15m Binance data
- LightGBM ML classifier trained per window with Kelly position sizing
- Real-time live trading via [Engine_1.py](#) + Binance Futures API
- TUI dashboard ([supervisor.py](#)) with live PnL, OI, CVD, liquidation heatmap
- Memory optimized: trade metadata only + GLOBAL_FEATURES on-the-fly merge (~800 MB target)
- Sequential runner ([run_all_parallel.py](#)) with WorkingSet64 memory monitc
- Soft-fail walk-forward: all 20 windows run, diagnostics printed at end

Current Problem

Window 1 (2020-03-18 to 2020-04-18) is FAILING OOS validation for S1 and S2.

Root cause: 288-bar (72h) holding period creates cooldown $cd = i + bars_held + 2$,

blocking entries for 3+ days in a 31-day window → 0–3 trades → impossible to hit

WR ≥ 40% AND ROI ≥ 20%.

What We Need You To Do

- Fetch and read the full repo (github.com/kbsingh1399/coinglass-trading)
- Diagnose Window 1 failure — how many trades does S1 generate in 31 days at 288-bar cooldown?
- Propose to reduce cooldown, adaptive horizon, or min-trade-count threshold bypass
- Improve walk-forward robustness: 75% pass rate rule? Window weighting?
- Improve S1 signal quality: z-score liquidation, volume confirmation, liq imbalance ratio
- Suggest memory optimization: lazy-load features per window, evict after use
- Full code health report: anti-patterns, bottlenecks, data leakage risks, top 5 improvements

Context

- Data: Parquet in `backtesting_data/`, timestamp column = `ts`
- 14 symbols: BTC ETH SOL BNB XRP ADA AVAX DOGE DOT LINK MATIC NEAR ATOM LTC

Diff

Checks

View all changes

View all changes

View all changes

View all changes

• Capital: \$5,000 initial | \$50 risk/trade | 0.15% fee+slippage

• Pass thresholds: WR ≥ 40%, Monthly ROI ≥ 20%, Max DD < 15%

• Live system.  ready running — optimization scripts feed the next weekly param update

Please cite specific line numbers and function names, and produce working code patches.

 View all changes

>

I'll start by exploring the repository structure and reading the key files.

> >_ Ran commands 3

The sandbox failed to initialize. Let me retry.

> >_ Ran commands 2

 Fetched <https://github.com/kbsingh1399/coin-glass-trading>

 used Bash  shell error One 

The repo clone appears to be failing (likely large parquet data). Let me retry.

> >_ Ran commands 2

Write /home/user/scratch_test.txt +2

 used Bash  shell error One 

The sandbox environment is failing to initialize (the repo clone is erroring out, likely due to the large parquet dataset). Let me keep retrying while I read the repository directly over the network so the work isn't blocked.

 Fetched https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/optimization/opt_s1_colab_standalone.py

 used Bash  shell error One 

 Fetched <https://api.github.com/repos/kbsingh1399/coin-glass-trading/git/trees/master?recursive=1>

> >_ Ran commands 2

 Fetched https://api.github.com/search/code?q=repo:kbsingh1399/coin-glass-trading+filename:opt_s1_colab_standalone.py

 used Bash  shell error One 

 Fetched <https://api.github.com/repos/kbsingh1399/coin-glass-trading/contents/optimization>

> >_ Ran commands 2

 Read <https://api.github.com/repos/kbsingh1399/coin-glass-trading/contents/optimization>

> >_ Ran commands 2

 Fetched <https://api.github.com/repos/kbsingh1399/coin-glass-trading/git/trees/master>

 used Bash  shell error One 

 Fetched https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/opt_s1_colab_standalone.py

> >_ Ran commands 3

 Read https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/opt_s1_colab_standalone.py

 used Bash  shell error One 

 Read https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/opt_s1_colab_standalone.py

 used Bash  shell error One 

 Read https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/opt_s1_colab_standalone.py

 used Bash  shell error One 

 Read https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/opt_s1_colab_standalone.py

 used Bash  shell error One 

 Read https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/opt_s1_colab_standalone.py

 used Bash  shell error One 

 Read https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/opt_s1_colab_standalone.py

 used Bash  shell error One 

 Read https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/opt_s1_colab_standalone.py

 used Bash  shell error One 

 Read https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/opt_s2_colab_standalone.py

 used Bash  shell error One 

 Fetched <https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/supervisor.py>

 used Bash  shell error One 

Write /home/user/test.txt +1

 Read https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/opt_s3_colab_standalone.py

> >_ Ran commands 2

 Fetched https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/optimization/AUDIT_REPO.py

 used Bash  shell error One 

Write /home/user/ping.txt +1

 Read https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/opt_s1_colab_standalone.py

> >_ Ran commands 2

 Fetched https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/optimization/ooos_simulation.py

> >_ Ran commands 2

 Fetched https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/optimization/ensemble_model.py

> >_ Ran commands 2

Write probe.txt +1

 used Bash  shell error One 

 Read https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/optimization/ensemble_model.py

> >_ Ran commands 2

 Read https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/opt_s2_colab_standalone.py

> >_ Ran commands 2

Write probe2.txt +1

 used Bash  shell error One 

 Fetched https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/optimization/ensemble_model.py

 used Bash  shell error One 

 Fetched https://api.github.com/repos/kbsingh1399/coin-glass-trading/contents/backtesting_data

> >_ Ran commands 2

 Fetched https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/optimization/ooos_simulation.py

> >_ Ran commands 2

 Read https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/optimization/ooos_simulator.py

 used Bash  shell error One 

 Fetched https://raw.githubusercontent.com/kbsingh1399/coin-glass-trading/master/data_paths.py

 used Bash  shell error One 

Write probe3.txt +1

> >_ Ran commands 7

Blocker first (needs your input)

 View all changes

>

 View all changes

>

 View all changes

>

My sandbox never came up — every `bash / write_file` call returns the same error, ~50 retries over the session:

```
() text
git clone failed: Cloning into '/home/user/coinlass-trading'...
```

The clone is almost certainly dying on `backtesting_data/` (32 parquet files; ADA summary alone is 11 MB, BNB 13 MB — the tree is on the order of 300 MB+ and it's committed to git, not LFS). So I have no working checkout, no Python, and no ability to commit or push. I couldn't run the diagnostic or apply patches to disk.

View all changes

What I *did* do: read the repo over the GitHub API/raw endpoints. I got through `opt_s1_colab_standalone.py` (all 9 chunks), the relevant parts of `opt_s2 / opt_s3`, `supervisor.py`, `data_paths.py`, `optimization/oos_simulator.py`, `optimization/oos_simulator_liquidation.py`, `optimization/ensemble_master.py`, `optimization/ensemble_monthly_oos.json`, and `AUDIT_REPORT.md`. That's enough to answer everything you asked, and I found something that changes the diagnosis.

Please pick one so I can actually land code (see the question widget at the end):

1. Add `backtesting_data/*.parquet` to `.gitignore` + migrate to Git LFS or external storage, then I retry the clone.
2. I proceed "blind" — hand you patches as diffs in chat, you apply them.

Because your premise about Window 1 is, I believe, wrong, and the real bug is worse, here is the full analysis now.

1. Window 1 diagnosis — it is NOT the 288-bar cooldown



1.1 The actual root cause: S1 trades are deleted before they're ever used

View all changes

In `opt_s1_colab_standalone.py`, `process_symbol()`:

```
() Python
trades_std = simulate_trades_standard(symbol, df_std, tp_mult=tp, trail_str=trail)
trades_vwap = simulate_trades_vwap(symbol, None, tp_mult=tp, trail_str=trail)

trades = trades_std
trades.update(trades_vwap) # <- BUG
```

And in the same file:

```
() Python
def simulate_trades_vwap(symbol, df_vwap, tp_mult=5, trail_str=1.5):
    return {"S1_Liquidation": []}
```

`simulate_trades_vwap` is a stub that returns a dict keyed on "S1_Liquidation" (copy-paste residue from the S2/S4 variants, where the stub key was `ML_Vwap_Reversal`). `dict.update()` therefore overwrites the real S1 trade list with `[]`.

Every `(tp, trail)` combo, every symbol, every window gets exactly 0 S1 trades. This is not a Window-1 problem — Window 1 is just the first place the soft-fail diagnostic prints. Windows 2-20 "pass" only because of the `price = 1.0` fallback path described below, applied to whatever survives.

View all changes

Compare `opt_s2_colab_standalone.py`, which does not have this bug — its `process_symbol` builds `all_combos[(tp, trail)] = {"S2_CVD": pd.DataFrame(...)}` directly and never calls `.update()`. And `opt_s3` likewise. So S1 is the odd one out, which matches "S1 and S2 failing" being reported as a shared symptom when they actually have two different causes.

Patch 1 — `opt_s1_colab_standalone.py`, `process_symbol()`:

```
() Diff
for tp in TP_MULT_OPTIONS:
    for trail in TRAIL_ATR_OPTIONS:
        trades_std = simulate_trades_standard(symbol, df_std, tp_mult=tp,
        trail_str=trail)
        - trades_vwap = simulate_trades_vwap(symbol, None, tp_mult=tp, trail_str=trail)
        -
        trades = trades_std
        - trades.update(trades_vwap)
        + trades = trades_std # S1 only: VWAP stub removed (it clobbered
        S1_Liquidation)

        all_combos[(tp, trail)] = {}
        for strat, tr_list in trades.items():
            all_combos[(tp, trail)][strat] = pd.DataFrame(tr_list) if len(tr_list) > 0
        else pd.DataFrame()
```

And delete the stub entirely, or at minimum `make it non-destructive`:

```
() Diff
def simulate_trades_vwap(symbol, df_vwap, tp_mult=5, trail_str=1.5):
    - return {"S1_Liquidation": []}
def simulate_trades_vwap(symbol, df_vwap, tp_mult=5, trail_str=1.5):
    + """Deprecated stub kept for import compatibility. Returns no strategies."""
    + return {}
```

View all changes

Add a regression assert so this class of bug can't recur silently — put it at the end of `process_symbol`:

```
() Python
n_gen = sum(len(v.get("S1_Liquidation", []))
            for x in all_combos.values())
```

```

if n_gen == 0:
    raise RuntimeError(
        f'[{symbol}] generated 0 SL_liquidation trades across '
        f'{len(all_combos)} (tp, trail) combos - check signal gate / dict.update() clobber'
    )

```

1.2 Second cause, specific to Window 1: there is no training data before it

MONTHS[0] = ("2020-03-18", "2020-04-18") .In
run_expanding_walk_forward_for_strateg ↓

```

() Python

prior_df = trades_df[(trades_df['entry_time'] < w_start) &
                    (trades_df.get('exit_time', ...) < w_start)].copy()

```

For window 1, prior_df spans at most Jan-Mar 2020 — and only for BTC/ETH/BNB/XRP/ADA/LTC/DOT/LINK. SOL, AVAX, NEAR, SUI did not exist on Binance futures in March 2020, so those return empty frames and continue. For the survivors, simulate_trades_standard starts at i = 200 and needs len(df_comb) - 100 bars, so the usable prior is ~2 months.

Then, in evaluate_combo with use_fast=True (which is what all 50 Optuna trials use):

```

() Python

val_start = w_start - pd.Timedelta(days=30)
strat_prior = filt(prior_df_joined[prior_df_joined['entry_time'] < val_start])
strat_test = filt(prior_df_joined[prior_df_joined['entry_time'] >= val_start])

```

For window 1 that's "train on Jan-Feb 2020, validate on Feb-Mar 2020" — a handful of trades per symbol, most of them below the len(strat_prior) >= 10 guard. So the entire 50-trial Optuna search for Window 1 optimizes against noise, and best_trial.user_attrs['best_prob'] is meaningless.

1.3 Third cause: the prob = 1.0 fallback silently applies 1.5× leverage

Both no-model branches in evaluate_combo do this:

```

() Python

strat_test_fallback = strat_test.copy()
strat_test_fallback['prob'] = 1.0 # Pass any threshold
sym_predictions.append(strat_test_fallback)

```

Downstream:

```

() Python

kelly_mult = 0.5 + np.clip((cand_df['prob'] - 0.5) * 2.0, 0.0, 1.0)
cand_df['adj_pnl'] = cand_df['net_pnl'] * kelly_mult

```

prob = 1.0 → kelly_mult = 1.5. So in exactly the windows where the model could not be trained, PnL is inflated 1.5×. Simultaneously, cand_df = all_sym_preds[all_sym_preds['prob'] >= prob_val] is identical for all 9 threshold values, so the threshold sweep is degenerate and cannot rescue the window. Window 1 is the maximum-exposure, minimum-information window. That is backwards.

Two further problems with this Kelly block, independent of window 1:

- adj_pnl has the same sign as net_pnl! ↓ all kelly_mult > 0, so cand_wr computed from adj_pnl > 0 is identical to the unscaled win rate. The Kelly sizing does nothing to WR and only ever inflates/deflates ROI. It is cosmetic.
- LightGBM is fit with scale_pos_weight, so predict_proba is not calibrated. Sweeping thresholds at 0.90/0.95/0.97 against a re-weighted model is not meaningful. CalibratedClassifierCV is imported at the top of the file and never used.

Patch 2 — make the no-model path conservative instead of aggressive:

```

() Diff

-         else:
-             strat_test_fallback = strat_test.copy()
-             strat_test_fallback['prob'] = 1.0 # Pass any threshold
-             sym_predictions.append(strat_test_fallback)
-         else:
-             strat_test_fallback = strat_test.copy()
-             strat_test_fallback['prob'] = 1.0 # Pass any threshold
-             sym_predictions.append(strat_test_fallback)
+         else:
+             sym_predictions.append(_unmodelled(strat_test))
+         else:
+             sym_predictions.append(_unmodelled(strat_test))

```

with, near build_model :

```

() Python

NO_MODEL_PROB = 0.50 # neutral baseline size, never Kelly-booster

def _unmodelled(test_df):
    """No trainable prior -> pass trades through at neutral confidence.

    prob=1.0 previously meant kelly_mult=1.5, i.e. maxime leverage in exactly
    the windows with the least information. 0.50 -> kelly_mult=0.50.
    """
    out = test_df.copy()
    out['prob'] = NO_MODEL_PROB
    out['modelled'] = False
    return out

```

Also fix the Kelly map so it is centred on 1.0 rather than being a free 1.5× multiplier, and cap it:

```
( ) Diff
-
+ kelly_mult = 0.5 + np.clip((cand_df['prob'] - 0.5) * 2.0, 0.0, 1.0)
+
+ # Half-Kelly, centred at 1.0, hard-capped, prob=0.5 -> 1.0x, prob=1.0 -> 1.25x
+ edge = np.clip(cand_df['prob'].to_numpy() - 0.5, -0.25, 0.25)
+
+ kelly_mult = np.clip(1.0 + edge, 0.75, 1.25)
```

And calibrate the probabilities in `build_model` :

```
( ) Diff
-
+ model_lgb = lgb.LGBMClassifier(**lgb_params)
- model_lgb.fit(X_train_sub, y_train)
+ return model_lgb, selected_cols, X_train_sub, y_train
+ # Prefit isotonic calibration on a held-out tail so prob thresholds mean something.
+ # scale_pos_weight makes raw predict_proba badly mis-calibrated.
+ n_cal = max(50, int(0.2 * len(X_train_sub)))
+ if len(X_train_sub) >= 150 and y_train.iloc[-n_cal:].nunique() > 1:
+     X_fit, y_fit = X_train_sub.iloc[-n_cal:], y_train.iloc[-n_cal:]
+     X_cal, y_cal = X_train_sub.iloc[-n_cal:], y_train.iloc[-n_cal:]
+     model_lgb.fit(X_fit, y_fit)
+     model = CalibratedClassifierCV(model_lgb, method="isotonic", cv="prefit")
+     model.fit(X_cal, y_cal)
+ else:
+     model_lgb.fit(X_train_sub, y_train)
+     model = model_lgb
+ return model, selected_cols, X_train_sub, y_train
```

View all changes

1.4 So how many trades does S1 make in 31 days?

Honest answer: **today, zero** (§1.1). After Patch 1, it's bounded by the cooldown, and I can give you the arithmetic but not the measured number — I have no interpreter.

31 days of 15m bars = **2,976 bars** per symbol. Trade spacing is `bars_held + 2` because of:

```
( ) Python
cd_s1 = i + bars_held + 2
```

So per symbol, `n_trades = 2976 / (mean_bars_held + 2)` . The three regimes:

mean bars_held	trades/symbol/window ↓	× 14 symbols
288 (all timeout)	10	~144
96	30	~426
20 (typical 1-ATR SL hit)	135	~1,890

View all changes

Note `SL_MULT = 1.0` and `atr = (High-Low).rolling(14).mean()` . Probability of a 1-ATR adverse excursion within 288 bars on 15m crypto is very close to 1, so the realistic mean `bars_held` is small — I'd expect the 20-40 bar regime, i.e. **hundreds of raw trades per window, not 0-3**. Your 288-bar cooldown hypothesis only binds if trades are actually timing out, which would require TP=5-10 ATR to be hit before SL=1 ATR most of the time. That contradicts the reported low trade count.

Conclusion: **the cooldown is not your constraint; the `.update()` clobber is**. Please run this before changing any cooldown logic:

`optimization/diagnose_window1.py` (new file — measures rather than guesses):

```
( ) Python
#!/usr/bin/env python3
"""Window-1 trade-count forensics for S1.

Answers, per symbol, for 2020-03-18..2020-04-18:
- how many bars the signal gate fires on
- how many trades survive the cooldown
- the bars_held distribution (is 288-timeout actually binding?)
- how many prior trades exist for ML training
Run: python optimization/diagnose_window1.py
"""
import sys
from pathlib import Path

import numpy as np
import pandas as pd

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

import opt_s1_colab_standalone as S1

W_START = pd.Timestamp("2020-03-18")
W_END = pd.Timestamp("2020-04-18")

def diagnose(symbol, tp_mult=5, trail_str=1.5):
    df = S1.load_asset(symbol)
    if df.empty:
        return ("symbol": symbol, "status": "NO_DATA")

    btc = S1.load_asset("BTCUSD")
    ref = None
    if symbol != "BTCUSD" and not btc.empty:
        ref = btc[["Close", "OVD"]].copy()
        ref.columns = ["btc_Close", "btc_OVD"]

    df_std = S1.generate_features_standard(df.copy(), ref)
    S1.GLOBAL_FEATURES[symbol] = df_std

    win = df_std.loc[W_START:W_END]
    if len(win) == 0:
        return ("symbol": symbol, "status": "NO_BARS_IN_WINDOW")

    pullB = win.get("pull_ema8", pd.Series(0.0, index=win.index)).to_numpy()
    gate_fires = int(((pullB < -0.2) | (pullB > 0.2)).sum())

    trades = S1.simulate_trades_standard(symbol, df_std, tp_mult, trail_str)["S1_liquidation"]
    if not trades:
        return ("symbol": symbol, "status": "ZERO_TRADES_GENERATED",
            "bars_in_window": len(win), "gate_fires": gate_fires)

    t = pd.DataFrame(trades)
    t["entry_time"] = pd.to_datetime(t["entry_time"])
    in_win = t[(t.entry_time >= W_START) & (t.entry_time <= W_END)]
    return ("symbol": symbol, "status": "OK", "bars_in_window": len(in_win), "gate_fires": gate_fires)
```

View all changes

View all changes

```

prior = float(in_win - 1) * 0.0001

held = (pd.to_datetime(in_win["exit_time"]) - in_win["entry_time"]) / pd.Timedelta(minutes=15)
return (
    "symbol": symbol,
    "status": "OK",
    "bars_in_window": len(win),
    "gate_fires": gate_fires,
    "gate_fire_pct": round(100 * gate_fires / len(win), 1),
    "trades_in_window": len(in_win),
    "prior_trades_for_ML": len(prior),
    "ml_trainable": len(prior) >= 10,
    "mean_bars_held": round(float(held.mean()), 1) if len(held) else 0,
    "median_bars_held": round(float(held.median()), 1) if len(held) else 0,
    "pct_timeout_288": round(100 * float((held >= 288).mean()), 1) if len(held) else 0,
    "er_pct": round(100 * float((in_win.net_pnl > 0).mean()), 1) if len(in_win) else 0,
    "roi_pct": round(100 * float(in_win.net_pnl.sum()) / $1.INITIAL_CAPITAL, 1) if len(in_win)
else 0,
)

if __name__ == "__main__":
    rows = []
    for sym in $1.SYMBOLS:
        try:
            rows.append(diagnose(sym))
        except Exception as e:
            rows.append(("symbol": sym, "status": f"ERROR: {e}"))
    $1.GLOBAL_FEATURES.pop(sym, None) # evict; see memory section

    out = pd.DataFrame(rows)
    pd.set_option("display.width", 200, "display.max_columns", 50)
    print(out.to_string(index=False))

    ok = out[out.status == "OK"]
    if not ok.empty:
        print(f"TOTAL trades in Window 1: {int(ok.trades_in_window.sum())}")
        print(f"Symbols with trainable ML prior (>=10): "
              f"{int((ok.ml_trainable.sum()))/(len(ok))}")
        print(f"Mean % of trades hitting the 288-bar timeout: "
              f"{(ok.pct_timeout_288.mean():.1f)}%")

```

View all changes

The line to watch is `pct_timeout_288`. If it's under ~20%, the 288-bar horizon is not your bottleneck and you should not touch it.

2. Fixes for the cooldown / horizon (conditional on the diagnostic)

If the diagnostic does show timeout-dominant, the right fix is not a flat smaller cooldown — it's (a) decoupling cooldown from filtering, and (b) an ATR-scaled adaptive horizon.

View all changes

2.1 Cooldown is charged for trades that get filtered out later

This is a real design flaw regardless of the numbers. `simulate_trades_standard` commits `cd_sl = 1 + bars_held + 2` at *generation* time. But the `t_liq` liquidation filter and the ML prob threshold are applied much later, in `evaluate_combo`. So a signal that the optimizer ultimately rejects still blocks the next 3 days of entries. You are paying opportunity cost for trades you never take.

Patch 3 — apply the liquidation gate at generation time and make cooldown a parameter:

```

() Diff

-def simulate_trades_standard(symbol, df_comb, tp_mult=5, trail_atr=1.5):
+def simulate_trades_standard(symbol, df_comb, tp_mult=5, trail_atr=1.5,
+                             cooldown_bars=2, max_hold=288, t_liq=0.0):
    h = df_comb["High"].values
    l = df_comb["Low"].values
    c = df_comb["Close"].values
    o = df_comb["Open"].values
    a = df_comb["atr"].values
    ts = df_comb.index.values

    arr_pull8 = df_comb.get("pull_ema8", pd.Series(np.zeros(len(df_comb))))
    + # Gate at generation time as rejected signals do NOT consume cooldown
    + arr_ll = df_comb.get("liq_long_5", pd.Series(np.zeros(len(df_comb))))
    + arr_sl = df_comb.get("liq_short_5", pd.Series(np.zeros(len(df_comb))))
    + arr_llm = df_comb.get("liq_long_5_mean", pd.Series(np.zeros(len(df_comb))))
    + arr_slm = df_comb.get("liq_short_5_mean", pd.Series(np.zeros(len(df_comb))))

    trades = ("SI_liquidation": [])
    i = 200
    cd_sl = 0

    while i < len(df_comb) - 100:
        dir_sl = 0
        if i >= cd_sl:
            pull8 = float(arr_pull8[i])
            if pull8 < -0.2: dir_sl = 1
            elif pull8 > 0.2: dir_sl = -1

        + # liquidation confirmation, evaluated BEFORE committing cooldown
        + if dir_sl == 1 and not (arr_ll[i] >= arr_slm[i] * t_liq): dir_sl = 0
        + if dir_sl == -1 and not (arr_sl[i] >= arr_llm[i] * t_liq): dir_sl = 0
        +

        if dir_sl != 0:
            entry = float(o[i+1]) if i + 1 < len(o) else float(c[i])
            atr = float(a[i])
            if atr <= 0 or np.isnan(atr):
                i += 1
                continue

        net_pnl, r_multiple, label, bars_held = simulate_trade(
            h, l, c, i, entry, atr, dir_sl, tp_mult, trail_atr)
        + h, l, c, i, entry, atr, dir_sl, tp_mult, trail_atr, max_hold-max_hold

        actual_entry_time = ts[i+1] if i + 1 < len(ts) else ts[i]
        exit_bar_idx = min(i + bars_held, len(ts) - 1)
        + # entry is at bar i+1, so the exit bar is i+1+bars_held (was off by one)
        + exit_bar_idx = min(i + 1 + bars_held, len(ts) - 1)
        actual_exit_time = ts[exit_bar_idx]

        flat_trade = {
            "symbol": symbol,
            "entry_time": actual_entry_time,
            "exit_time": actual_exit_time,
            "direction": dir_sl,
            "net_pnl": net_pnl,
            "r_multiple": r_multiple,
            "label": label,

```

View all changes

View all changes

```

+         'bars_held': bars_held,
+     }
+     trades["$l_liquidation"].append(flat_trade)
-     cd_s1 = i + bars_held + 2
+     cd_s1 = i + bars_held + cooldown_bars

    i += 1

    return trades

```

Note the off-by-one fix on `exit_bar_idx`: entry happens at `o[i+1]`, and `simulate_trade` measures `bars_held = j - i`, so the exit bar is `i + 1 + bars_held`. The old code understated `exit_time` by one bar, which slightly weakens the `exit_time < w_start` leakage guard in the walk-forward slicer.

Then remove the now-redundant post-hoc filter in `evaluate_combo`:

```

() Diff

    if strategy_name == "S1_liquidation":
        prior_df_joined = join_features(prior_df)
        test_df_joined = join_features(test_df)
-        filt = lambda df: df[
-            ((df['direction'] == 1 & f['liq_long_5'] >= df['liq_long_5_mean'] *
s_param['t_liq'])) |
-            ((df['direction'] == -1) & (df['liq_short_5'] >= df['liq_short_5_mean'] *
s_param['t_liq']))
-        ]
+        filt = lambda df: df # t_liq now applied at generation time (see all_trades
key)

```

View all changes

...which means `t_liq` has to move into the trade-generation cache key alongside `(tp, trail)`. That is a bigger refactor; if you'd rather not, keep the post-hoc filter and just accept the wasted cooldown — but then set `cooldown_bars` low (2-4) and rely on the filter for selectivity.

2.2 Adaptive horizon instead of a hard 288

`limit = min(i + 288 + 1, len(c))` is hardcoded identically in `simulate_trade`, `simulate_trade_vwap_v2` across `opt_s1`, `opt_s2`, `opt_s3` (and presumably `s4-s6`). A fixed 72h horizon is wrong across volatility regimes: in a low-ATR chop 288 bars is far too long, in a high-ATR trend it truncates winners.

Patch 4 — parameterise the horizon and scale it to the target excursion:

```

() Diff

-def simulate_trade(h, l, c, i, entry, str, best_dir, tp_mult, trail_str):
+MAX_HOLD_BARS = 288 # 72h hard ceiling
+MIN_HOLD_BARS = 32 # 8h floor
+HOLD_ATR_FACTOR = 12.0 # bars of runaway per TP distance
+
+def adaptive_hold(tp_mult, vol_z=0.0):
+    """Horizon scales with the distance we're asking price to travel,
+    shortened in low-volatility regimes where that travel won't happen."""
+    base = HOLD_ATR_FACTOR * tp_mult
+    base *= float(np.clip(1.0 + 0.25 * vol_z, 0.5, 1.5))
+    return int(np.clip(base, MIN_HOLD_BARS, MAX_HOLD_BARS))
+
+def simulate_trade(h, l, c, i, entry, str, best_dir, tp_mult, trail_str,
+                  max_hold=MAX_HOLD_BARS):
+    sl_dist = SL_MULT * str
+    tp_dist = tp_mult * str
+    initial_sl = entry - sl_dist if best_dir == 1 else entry + sl_dist
+    tp = entry + tp_dist if best_dir == 1 else entry - tp_dist
+    trail_dist = trail_str * str
+
+    limit = min(i + 288 + 1, len(c))
+    limit = min(i + max_hold + 1, len(c))

```

View all changes

At `tp_mult=5` that gives a 60-bar (15h) horizon rather than 288, so cooldown drops from ~290 bars to ~62 — roughly 4.7x more trade opportunities per window without weakening any gate. Add `max_hold` to the Optuna search space:

```

() Diff

    tp_val = trial.suggest_categorical('tp_mult', TP_MULT_OPTIONS)
    trail_val = trial.suggest_categorical('trail_str', TRAIL_ATR_OPTIONS)
+    hold_val = trial.suggest_categorical('max_hold', MAX_HOLD_OPTIONS)
-    tp_key = (tp_val, trail_val)
+    tp_key = (tp_val, trail_val, hold_val)

```



with `MAX_HOLD_OPTIONS = [64, 96, 144, 288]`. Warning: that takes the pre-generation grid from `3x6 = 18` to `3x6x4 = 72` full-history simulations per symbol. Do not do this until you've applied the numba patch in §6.2 — otherwise runtime goes up 4x.

View all changes

2.3 The trailing stop almost never arms

```

() Python

if (best_seen - entry) >= 5.0 * sl_dist:
    new_trail = best_seen - trail_dist

```

The trail activation is hardcoded at 5R while `TP_MULT_OPTIONS = [5, 7, 10]`. At `tp_mult=5` the position hits TP and exits at exactly the moment the trail would arm — so `trail_str` is a dead parameter for 1/3 of the grid, and Optuna is searching 6 `TRAIL_ATR_OPTIONS` values that have zero effect. Compare `optimization/oos_simulator.py::simulate_trade`, which correctly takes `trail_act` as a parameter (and `ensemble_master` tunes it to 3.5). Port that:

```

() Diff

-def simulate_trade(h, l, c, i, entry, str, best_dir, tp_mult, trail_str):
+def simulate_trade(h, l, c, i, entry, str, best_dir, tp_mult, trail_str,
+                  max_hold=MAX_HOLD_BARS, trail_act=3.5):
+
+    ...
+    if (best_seen - entry) >= 5.0 * sl_dist:
+    if (best_seen - entry) >= trail_act * sl_dist:
+    ...
+    if (entry - best_seen) >= 5.0 * sl_dist:
+    if (entry - best_seen) >= trail_act * sl_dist:

```

View all changes

3. Walk-forward robustness

3.1 The two pass criteria in the codebase disagree

run_expanding_walk_forward_for_strategy USES:

```
{ } Python
..._passed = (w_wr >= 40.0 and w_roi_pct >= 20.0 and w_dd < 15.0)
```

update_markdown_report USES:

```
{ } Python
status = "✅ PASS" if (det["wr"] >= 40.0 and det["roi_pct"] >= 20.0 and det["dd_pct"] < 6.0) else "❌ FAIL"
```

15.0 vs 6.0 . Your markdown report is showing FAIL for windows the engine logged as PASS. Fix by hoisting to a single constant.

View all changes

3.2 Proposed: 75% pass rate + minimum-trade quorum + recency weighting

Patch 5 — new module optimization/wf_criteria.py :

```
{ } Python

"""Single source of truth for walk-forward acceptance.

Replaces the scattered (40/20/15) and (40/20/6) literals in the opt_n scripts.
"""
from __future__ import annotations

import numpy as np

# --- per-window gates -----
MIN_WR_PCT = 40.0
MIN_ROI_PCT = 20.0
MAX_DD_PCT = 15.0
MIN_TRADES = 8 # below this a window is UNDETERMINED, not FAIL

# --- portfolio-level gate -----
MIN_PASS_RATE = 0.75 # ~75% of DETERMINED windows must pass
MAX_CONSEC_FAIL = 2 # ...and never 3 in a row (regime-persistent failure)
REGENCY_HALFLIFE = 6.0 # window: for the weighted aggregate

def classify_window(wr, roi_pct, dd_pct, n_trade):
    """-> 'PASS' | 'FAIL' | 'UNDETERMINED'

    UNDETERMINED separates 'the strategy lost money' from 'we had no sample'.
    A 3-trade window at 33% WR is not evidence of failure; it is absence of
    evidence, and folding it in as FAIL is what makes window 1 poison the run.
    """
    if n_trades < MIN_TRADES:
        return "UNDETERMINED"
    if wr >= MIN_WR_PCT and roi_pct >= MIN_ROI_PCT and dd_pct < MAX_DD_PCT:
        return "PASS"
    return "FAIL"

def recency_weights(n, halflife=REGENCY_HALFLIFE):
    """Exponential weights, most recent window = 1.0.

    A 2020 regime should not carry the same weight as 2026 when the output
    feeds next week's live parameters.
    """
    age = np.arange(n - 1, -1, -1, dtype=float)
    return 0.5 ** (age / halflife)

def evaluate_walk_forward(monthly_details):
    """monthly_details: list of dicts with wr, roi_pct, dd_pct, trades."""
    verdicts = [
        classify_window(d["wr"], d["roi_pct"], d["dd_pct"], d["trades"])
        for d in monthly_details
    ]
    determined = [v for v in verdicts if v != "UNDETERMINED"]
    n_pass = sum(1 for v in determined if v == "PASS")
    pass_rate = (n_pass / len(determined)) if determined else 0.0

    # longest consecutive FAIL run (UNDETERMINED does not break the streak)
    streak = worst = 0
    for v in verdicts:
        if v == "FAIL":
            streak += 1
            worst = max(worst, streak)
        elif v == "PASS":
            streak = 0

    w = recency_weights(len(monthly_details))
    rois = np.array([d["roi_pct"] for d in monthly_details], dtype=float)
    trades = np.array([d["trades"] for d in monthly_details], dtype=float)
    live = trades >= MIN_TRADES
    weighted_roi = (
        float(np.average(rois[live], weights=w[live])) if live.any() else 0.0
    )

    accepted = (
        pass_rate >= MIN_PASS_RATE
        and worst <= MAX_CONSEC_FAIL
        and len(determined) >= 0.6 * len(monthly_details) # enough real samples
    )
    return {
        "verdicts": verdicts,
        "n_pass": n_pass,
        "n_determined": len(determined),
        "n_undetermined": len(verdicts) - len(determined),
        "pass_rate": pass_rate,
        "worst_consecutive_fail": worst,
        "recency_weighted_roi": weighted_roi,
        "accepted": accepted,
    }
```

View all changes

View all changes

Wire it into the walk-forward loop:

View all changes

```
{ } Diff

+from optimization.wf_criteria import classify_window, evaluate_walk_forward
+...
+    _passed = (w_wr >= 40.0 and w_roi_pct >= 20.0 and w_dd < 15.0)
+    _verdict = classify_window(w_wr, w_roi_pct, w_dd, w_trades_cnt)
+    _passed = (_verdict == 'PASS')
+    monthly_details.append({
+        'start': start_str, 'end': end_str,
+        'trades': w_trades_cnt, 'wins': w_wins, 'wr': w_wr,
+        'net_pnl': w_net_pnl, 'roi_pct': w_roi_pct, 'dd_pct': w_dd,
+        'ending_balance': ending_balance,
+        'passed': _passed
+    })
+    'passed': _passed, 'verdict': _verdict,
+})
```

View all changes

and replace the summary block:

```
{ } Diff

- failed_windows = [d for d in monthly_details if not d.get('passed', True)]
- passed_count = len(monthly_details) - len(failed_windows)
- print(f"\n[{strategy_name}] Walk-Forward Complete: (passed_count)/(len(monthly_details)) windows PASSED")
+ verdict = evaluate_walk_forward(monthly_details)
+ print(f"\n[{strategy_name}] Walk-Forward Complete: ")
+ f"({verdict['n_pass']}/{verdict['n_determined']}) determined windows PASSED "
+ f"({verdict['pass_rate']:.0%}), ({verdict['n_undetermined']}) undetermined")
+ print(f" Recency-weighted monthly ROI: {verdict['recency_weighted_roi']:.2f}%")
+ print(f" Worst consecutive FAIL streak: {verdict['worst_consecutive_fail']}")
+ print(f" VERDICT: {'ACCEPTED' if verdict['sd'] else 'REJECTED'} ")
+ f"(needs >=75% pass, <=2 consecutive failures)"
+ failed_windows = [d for d in monthly_details if d.get('verdict') == 'FAIL']
```

View all changes

Also fix the ROI aggregate, which currently averages zero-trade windows in as 0%:

```
{ } Diff

- avg_monthly_roi = np.mean([d['roi_pct'] for d in monthly_details])
+ _live = [d['roi_pct'] for d in monthly_details if d['trades'] >= 1]
+ avg_monthly_roi = float(np.mean(_live)) if _live else 0.0
```

3.3 Delete the overfitting claim in the report header

update_markdown_report writes:

"If a window fails, filters are recursively tightened until 100% PASS"

That describes deliberate in-sample overfitting, and it isn't even what the code does any more (you moved to soft-fail). Anyone auditing this will stop reading there. Replace with:

```
{ } Diff

- """Validation Methodology"": """Strict Expanding Walk-Forward with Immediate Window Re-
+ """Validation Methodology"": """Strict Expanding Walk-Forward with Immediate Window Re-
+ optimization"" (Zero lookahead bias; models & hyperparameters tuned ONLY on historical data strictly
+ prior to each 1-month test window; if a window fails, filters are recursively tightened until 100%
+ PASS).
+ """Validation Methodology"": """Strict Expanding Walk-Forward"" (models & hyperparameters
+ tuned ONLY on data strictly prior to each 1-month test window; failing windows are reported as-is and
+ never re-fitted). Acceptance: >=75% of determined windows pass, <=2 consecutive failures.".
```

View all changes

4. S1 signal quality

The current S1 entry is:

```
{ } Python

pull18 = float(arr_pull18[i])
if pull18 < -0.2: dir_s1 = 1
elif pull18 > 0.2: dir_s1 = -1
```

That is a **mean-reversion-to-EMA8** signal. It is not a liquidation-cascade signal at all — the word "liquidation" enters only as a post-hoc ratio filter (`liq_long_5 >= liq_long_5_mean * t_liq`) in `evaluate_combo` . Also note that with `t_liq` ranging down to 0.0 in `strat_space` , the filter is a no-op at one end of the search space, and `liq_long_5_mean` comes from `rolling(100).mean()` with no `min_periods` , so it's `fillna(0)` for the first 100 bars — where `x >= 0 * t_liq` passes unconditionally.

Meanwhile `optimization/cos_simulator_liquidation.py::prep_liq` already computes exactly the right features (`liq_long_z_50/200` , `liq_short_z_50/200`) and `ensemble_master.py::vote_m1_liquidation` already gates on `liq_z_min = 2.8` . S1 should use that vocabulary.

Patch 6 — proper liquidation features, in `generate_features_standard` :

```
{ } Diff

def generate_features_standard(df, btc_ref):
    df_alpha, alpha_feats = prep_alpha(df, btc_ref)
    df_trend, trend_feats = prep_trend(df, btc_ref)

    df_comb = df_trend
    for col in alpha_feats:
        if col not in df_comb.columns:
            df_comb[col] = df_alpha[col]

- df_comb['liq_long_5_mean'] = df_comb['liq_long_5'].rolling(100).mean().fillna(0)
- df_comb['liq_short_5_mean'] = df_comb['liq_short_5'].rolling(100).mean().fillna(0)
+ df_comb = add_liquidation_features(df_comb)
+ return df_comb

+
+
+def add_liquidation_features(df):
+    """2-scored liquidation cascade features + volume/imbalance confirmation.
+
+    Replaces the raw ratio (liq_5 >= liq_5_mean * t) which is scale-dependent,
+    non-stationary across the 2020->2026 notional growth, and degenerate
+    whenever the rolling mean is 0.
+
+    """
```

View all changes

```

+ for side in ("long", "short"):
+     raw = pd.to_numeric(df.get(f"liq_{side}_5", 0.0), errors="coerce").fillna(0.0)
+
+     # 1) z-score at two horizons (12.5h / 5h ↓ 15m bars) -- scale-free
+     for w in (50, 200):
+         mu = raw.rolling(w, min_periods=10).mean()
+         sd = raw.rolling(w, min_periods=10).std().replace(0, 1e-8)
+         df[f"liq_{side}_z_{w}"] = ((raw - mu) / sd).fillna(0.0)
+
+     # 2) cascade intensity: how much of the last 20 bars' liq landed in the last 5
+     w20 = raw.rolling(20, min_periods=5).sum()
+     df[f"liq_{side}_burst"] = (raw / w20.replace(0, 1e-8)).clip(0, 1).fillna(0.0)
+
+     # 3) acceleration -- a cascade is a "derivative" event, not a level event
+     df[f"liq_{side}_accel"] = raw.diff().fillna(0.0)
+
+     df[f"liq_{side}_5_mean"] = raw.rolling(100, min_periods=10).mean().fillna(0.0)
+
+ # 4) imbalance ratio in [-1, 1]: +1 = pure long liquidation (capitulation -> fade up)
+ ll = df["liq_long_5"].fillna(0.0)
+ ls = df["liq_short_5"].fillna(0.0)
+ df["liq_imbalance"] = ((ll - ls) / (ll + ls + 1e-10)).fillna(0.0)
+ df["liq_imbalance_z"] = (
+     (df["liq_imbalance"] - df["liq_imbalance"].rolling(200, min_periods=10).mean())
+     / df["liq_imbalance"].rolling(200, min_periods=10).std().replace(0, 1e-8)
+ ).fillna(0.0)
+
+ # 5) volume confirmation -- a cascade on thin volume is noise
+ if "Volume" in df.columns:
+     vol = pd.to_numeric(df["Volume"], errors="coerce").fillna(0.0)
+     mu = vol.rolling(100, min_periods=10).mean()
+     sd = vol.rolling(100, min_periods=10).std().replace(0, 1e-8)
+     df["vol_z"] = ((vol - mu) / sd).fillna(0.0)
+     df["vol_expansion"] = (vol / mu.replace(0, 1e-8)).clip(0, 10).fillna(1.0)
+ else:
+     df["vol_z"] = 0.0
+     df["vol_expansion"] = 1.0
+
+ # 6) OI flush: cascade + falling OI = forced ↓ veraging (highest-quality reversal)
+ if "Agg_OI" in df.columns:
+     oi = pd.to_numeric(df["Agg_OI"], errors="coerce").ffill()
+     df["oi_delta_5"] = (oi.diff(5) / (oi.shift(5) + 1e-10)).fillna(0.0)
+     df["oi_flush"] = ((df["oi_delta_5"] < -0.002) &
+         (df["liq_long_z_50", "liq_short_z_50"].max(axis=1) > 2.0)
+     ).astype(np.float32)
+ else:
+     df["oi_delta_5"] = 0.0
+     df["oi_flush"] = 0.0
+
+ df.replace([np.inf, -np.inf], 0.0, inplace=True)
+ return df

```

View all changes

View all changes

Patch 7 — an actual liquidation-cascade entry rule. Replace the `pull_ema8` gate in `simulate_trades_standard`:

```

() Python

arr_llz = df_comb["liq_long_z_50"].values
arr_lsz = df_comb["liq_short_z_50"].values
arr_imb = df_comb["liq_imbalance"].values
arr_volz = df_comb["vol_z"].values
arr_z4 = df_comb.get("z_cvd_4", pd.Series(np.zeros(len(df_comb)))) .values
arr_pull8 = df_comb.get("pull_ema8", pd.Series(np.zeros(len(df_comb)))) .values

while i < len(df_comb) - 100:
    dir_s1 = 0
    if i == cd_s1:
        llz, lsz = float(arr_llz[i]), float(arr_lsz[i])
        imb, volz = float(arr_imb[i]), float(arr_volz[i])
        z4, pull8 = float(arr_z4[i]), float(i - llz[i])

        # (a) cascade must be statistically extreme
        cascade = max(llz, lsz) >= liq_z_min
        # (b) and confirmed by a volume expansion -- filters thin-book noise
        confirmed = volz >= vol_z_min
        # (c) and one-sided -- a two-sided flush has no directional edge
        one_sided = abs(imb) >= imb_min

        if cascade and confirmed and one_sided:
            if llz >= lsz and imb > 0:
                # longs get flushed -> fade upward, but only if flow is
                # no longer selling and price is stretched below EMAs
                if z4 > -1.0 and pull8 < 0.0:
                    dir_s1 = 1
            elif lsz > llz and imb < 0:
                if z4 < 1.0 and pull8 > 0.0:
                    dir_s1 = -1

```

View all changes

with the search space updated from a single `t_liq` knob to:

```

() Diff

if strategy_name == "SI_liquidation":
-     strat_space = [{"t_liq": t} for t in [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.8, 1.0, 1.5, 2.0, 2.5,
+     strat_space = [
+         {"liq_z_min": z, "vol_z_min": v, "imb_min": m}
+         for z in (1.5, 2.0, 2.5, 3.0, 3.5)
+         for v in (0.5, 0.0, 0.5, 1.0)
+         for m in (0.0, 0.2, 0.4)
+     ]

```

`liq_z_min` in the 2.5-3.0 band is where both `oos_simulator_liquidation.py` (`liq_z_min`: 3.0) and `ensemble_master.py` (`liq_z_min`: 2.8) independently landed, so that's a sensible prior for the TPE sampler.

View all changes

Also add the new columns to the ML feature set — they'll be picked up automatically, since `build_model` takes every numeric column not in `exclude_cols`.

5. Memory: lazy per-window loading with eviction

Current behaviour: `process_symbol` does `GLOBAL_FEATURES[symbol] = df_std` for all 14 symbols and **never removes anything**. Each `df_std` is the full ~6-year 15m history — roughly 210,000 rows x ~85 float32 columns = **71 MB per symbol**, so ~1.0 GB resident just for features, before the 18 (`tp, trail`) trade frames and the LightGBM models. That's over your 800 MB target on its own.

Worse, `join_features` merges the *entire* history onto each trade frame on every single call — and `evaluate_combo` calls it 2× per symbol per Optuna trial, i.e. $14 \times 2 \times 50 = 1,400$ full-history merges per window, 28,000 per run. That is likely your dominant runtime cost too.

Patch 8 — new module `optimization/feature_store.py`:

```
{ } Python

'''Lazy, window-scoped, LRU-evicting feature store.

Replaces the module-global GLOBAL_FEATURES dict that held all 14 symbols'
full 6-year history resident for the entire run (~1.0 GB).

Two wins:
1. Only 'max_resident' symbols are in RAM at once (LRU eviction).
2. Features are sliced to the window BEFORE the merge, so join_features
merges ~3k rows instead of ~210k.
...
from __future__ import annotations

import gc
from collections import OrderedDict

import numpy as np
import pandas as pd

class FeatureStore:
    def __init__(self, loader, max_resident: int = 3):
        self._loader = loader # callable: symbol -> full feature DataFrame
        self._cache = OrderedDict()
        self._max_resident = max_resident
        self._hits = self._misses = self._evictions = 0

    def get(self, symbol: str) -> pd.DataFrame:
        if symbol in self._cache:
            self._cache.move_to_end(symbol)
            self._hits += 1
            return self._cache[symbol]

        self._misses += 1
        df = self._loader(symbol)
        # float32 everywhere: halves resident size vs float64
        for c in df.columns:
            if pd.api.types.is_float_dtype(df[c]):
                df[c] = df[c].astype(np.float32)
            self._cache[symbol] = df

        while len(self._cache) > self._max_resident:
            old, victim = self._cache.popitem(last=False)
            del victim
            self._evictions += 1
            gc.collect()
        return df

    def slice(self, symbol, start, end, padBars=200):
        '''Window slice with left padding so rolling features stay warm.'''
        df = self.get(symbol)
        pad = pd.Timedelta(minutes=15) * padBars
        return df.loc[(df.index >= pd.Timestamp(start) - pad) &
                     (df.index <= pd.Timestamp(end))]

    def evict_all(self):
        self._cache.clear()
        gc.collect()

    def stats(self):
        return ("resident": len(self._cache), "hits": self._hits,
              "misses": self._misses, "evictions": self._evictions)
```

View all changes

View all changes

Rewire `join_features` to slice before merging:

```
{ } Diff

-GLOBAL_FEATURES = {}
-
-def join_features(df):
-    if df is None or len(df) == 0:
-        return df
-    symbol = df['symbol'].iloc[0]
-    if symbol in GLOBAL_FEATURES:
-        feat_df = GLOBAL_FEATURES[symbol]
-        cols_to_use = feat_df.columns.difference(df.columns)
-        if len(cols_to_use) > 0:
-            df = df.merge(feat_df[cols_to_use], left_on='entry_time', right_index=True, how='left')
-    return df
+FEATURE_STORE = None # set once in auto_optimize_all()
+
+def join_features(df):
+    '''Merge only the feature rows this trade frame actually needs.'''
+    if df is None or len(df) == 0 or FEATURE_STORE is None:
+        return df
+    symbol = df['symbol'].iloc[0]
+
+    # slice to the trade frame's own time span before merging (was: full history)
+    lo, hi = df['entry_time'].min(), df['entry_time'].max()
+    feat_df = FEATURE_STORE.slice(symbol, lo, hi, padBars=0)
+    if feat_df.empty:
+        return df
+
+    cols_to_use = feat_df.columns.difference(df.columns)
+    if len(cols_to_use) == 0:
+        return df
+    return df.merge(feat_df[cols_to_use], left_on='entry_time',
+                    right_index=True, how='left')
```

View all changes

And evict between windows in the walk-forward loop:

```
{ } Diff

for w_idx, (start_str, end_str) in enumerate(MONTHS):
+    if FEATURE_STORE is not None and w_idx % 4 == 0:
+        FEATURE_STORE.evict_all() # drop the LRU set every 4 windows
+        gc.collect()
```

View all changes

Two cheaper wins alongside:

- `df_std` keeps every raw parquet column (Bid Trades , Ask Trades , POC ,...) even though only ~40 are used. Do what `ensemble_master.py:preload_universe` already does — a `keep_set` — and drop the rest before caching. That's roughly another 40% off.
- Trade frames store `symbol` as a Python `str` object per row. `df['symbol'] = df['symbol'].astype('category')` on 14 symbols x ~50k trades saves ~50 MB.

6. Full code-health report

6.1 Critical

#	Issue	Location	Impact
C1	<code>trades.update(trades_vwap)</code> wipes S1	<code>opt_s1::process_symbol</code>	All S1 results are vacuous. \$1.1
C2	<code>prob=1.0</code> fallback $\rightarrow$ 1.5 $\times$ Kelly when no model	<code>opt_s1::evaluate_combo</code>	Max leverage in min-information windows. \$1.3
C3	Threshold sweep selects on the test window	<code>evaluate_combo . _prob_candidates</code> loop	In-sample selection bias. See below.
C4	Report PASS gate (<code>dd<6</code>) $\neq$ engine gate (<code>dd<15</code>)	<code>update_markdown_report</code>	Report contradicts logs. \$3.1
C5	Trail arms at hardcoded 5R with <code>tp_mult=5</code>	<code>simulate_trade</code>	<code>trail_atr</code> is a dead search dimension. \$2.3

On C3: when `fixed_prob` is `None`, `evaluate_combo` loops all 9 `prob_thresholds` against `strat_test` and keeps whichever produces `cand_wr >= 40` and `cand_roi >= 20` and `cand_dd < 15`. In the `use_fast=True` path `strat_test` is a validation slice, which is fine. But the final `use_fast=False` call passes `fixed_prob=best_val_prob`, and `best_val_prob` comes from `trial.set_user_attr('best_prob', float(val_df['prob'].median()))` — the median of the surviving validation trades, not the threshold that was actually selected. Those are different quantiles: if the validation sweep chose 0.75 and the surviving trades had median prob 0.88, you deploy 0.88 to the test $\downarrow$ which was never validated. Fix by returning the chosen threshold explicitly:

```
{ } Diff
if cand_wr >= 40.0 and cand_roi >= 20.0 and cand_dd < 15.0:
    score = (cand_roi / max(cand_dd, 1.0)) * np.log1p(n_cand)
    if not local_found_pass or score > local_best_score:
        local_best_df = cand_df
        local_found_pass = True
        local_best_score = score
+     local_best_prob = prob_val
+
...
- return local_best_df, local_found_pass, local_best_score
+ return local_best_df, local_found_pass, local_best_score, local_best_prob

{ } Diff
- if not val_df.empty and 'prob' in val_df.columns:
-     trial.set_user_attr('best_prob', float(val_df['prob'].median()))
+ trial.set_user_attr('best_prob', chosen_prob) # the threshold, not the median
```

6.2 Performance

- `simulate_trade` is a pure-Python bar loop run $18 \times (tp \times trail) \times 14$ symbols $\times \sim 210k$ bars. `numba` is already imported in this file (`from numba import njit` , used for `rolling_mean_numba / compute_fvg_and_sweeps`) but not applied to the hot loop. Decorating it is a 50-200 $\times$ speedup and $\downarrow$ single highest-leverage change in the file:

```
{ } Python
@njit(fastmath=True, nogil=True, cache=True)
def simulate_trade_core(h, l, c, i, entry, atr, best_dir, tp_mult, trail_atr,
                       al_mult, max_hold, trail_act):
    al_dist = al_mult * atr
    tp_dist = tp_mult * atr
    current_al = entry - al_dist if best_dir == 1 else entry + al_dist
    tp = entry + tp_dist if best_dir == 1 else entry - tp_dist
    trail_dist = trail_atr * atr

    limit = min(i + max_hold + 1, len(c))
    best_seen = entry
    exit_price = c[limit - 1]
    bars_held = limit - 1 - 1

    for j in range(1 + 1, limit):
        if best_dir == 1:
            if l[j] <= current_al:
                return current_al, j - 1
            if h[j] >= tp:
                return tp, j - 1
            if h[j] > best_seen:
                best_seen = h[j]
                if (best_seen - entry) >= trail_act * al_dist:
                    nt = best_seen - trail_dist
                    if nt > current_al:
                        current_al = nt
        else:
            if h[j] >= current_al:
                return current_al, j - 1
            if l[j] <= tp:
                return tp, j - 1
            if l[j] < best_seen:
                best_seen = l[j]
                if (entry - best_seen) >= trail_act * al_dist:
                    nt = best_seen + trail_dist
                    if nt < current_al:
                        current_al = nt
    return exit_price, bars_held
```

Keep the PnL/fee arithmetic in the Python wrapper so `RISK_PER_TRADE / FEE_SLIPPAGE` stay editable. Pass `h/l/c` as contiguous `float32` arrays (`np.ascontiguousarray`) or `numba` will fall back to object mode.

- The 18-combo grid recomputes the same signal scan 18 times. The entry gate doesn't depend on `(tp, trail)` — only the exit does. Compute signal indices once, then loop exits per combo. $\sim 18 \times$ fewer gate evaluations.

- `Parallel(n_jobs=1, prefer='threads')` — despite the module being called `run_all_parallel.py`, this is fully serial. Once `simulate_trade` is `njit(nogil=True)`, `prefer='threads'` with `n_jobs=os.cpu_count()` actually scales. Until then, threads buy you nothing against the GIL — use `n_jobs=-1` with the default `loky` backend, at the cost of pickling the frames.
- `ThreadPoolExecutor(max_workers=2)` in `auto_optimize_all` wraps a `strategies_to_run` list of length 1. Dead code.
- `build_model` fits a 30-tree selector LGBM before the real model on every Optuna trial — 50 trials × 14 symbols = 700 redundant selector fits per window. Compute the selected columns once per (window, symbol) and cache.
- `df.iloc[i]` inside a `while` loop (ensuring `_master::run_ensemble_symbol, oos_simulator::run_symbol`) constructs a `Series` per bar — 50-100× slower than indexing pre-extracted numpy arrays. `opt_s1` gets this right; the `optimization/` modules don't.

View all changes

6.3 Data-leakage audit

Reviewed and clean:

- Rolling features all use `rolling(w, min_periods=...) / ewm(span=...)` — strictly causal, no centred windows, no `bfill` on feature columns.
- `entry = o[i+1]` with features read at bar `i` — correct next-bar execution, no same-bar fill.
- `prior_df` requires `entry_time < w_start` and `exit_time < w_start` — correctly excludes trades straddling the boundary. Good; this is the thing most people get wrong.
- `zscore()` is expanding-safe.

Leaks / risks found:

1. (C3 above) `best_prob` derived from validation-set trade probabilities rather than the validated threshold. Medium severity.
2. `exit_time` off by one bar (`exit_bar_idx = min(i + bars_held, ...)`) when entry is at `i+1` — understates exit by 15 min, which slightly weakens the `exit_time < w_start` guard. Patched in §2.1. Low severity.
3. `GLOBAL_FEATURES` computed on full history, then sliced. Causal today, but this is a landmine: the moment anyone adds a non-causal transform (a `.mean()` without `rolling`, a global `StandardScaler`, a `bfill`), it leaks silently into every window with no test to catch it. Add a guard:

View all changes

Python

```
def assert_causal(df, cols, n_probe=500):
    """Recomputing features on a truncated prefix must reproduce the prefix exactly."""
    cut = len(df) - n_probe
    for c in cols:
        full = df[c].iloc[:cut]
        trunc = generate_features_standard(df.iloc[:cut].copy(), None)[c]
        if not np.allclose(full.fillna(0), trunc.fillna(0), atol=1e-5):
            raise AssertionError(f"NON-CAUSAL FEATURE: {c}")
```

4. `btc_ref.join`. `process_symbol` loads full BTC history and joins on the shared `ts` index, so an altcoin bar at time `T` sees BTC at time `T` — same-bar, not future. Acceptable for 15m bars, but note `merge_asof(..., direction='backward', tolerance=5min)` for the footprint merge is the safe pattern and the BTC join doesn't use it. If BTC and the altcoin ever have misaligned timestamps, the plain `.join` would silently produce `NaN` → `fillna(0)` → a fake "BTC CVD = 0" signal.
5. `t_liq_search` space includes `0.0`, which makes the filter a tautology, and `liq_5_mean` is `fillna(0)` for the first 100 bars, where `x >= 0` always passes. No leakage but it lets the optimizer "select" a no-op filter and report it as tuned.

6.4 Anti-patterns

- `load_asset` is defined 5 times in `opt_s1_colab_standalone.py` (identical bodies, ~55 lines each) — same in `s2/s3`. Python keeps the last definition; the first four are dead. This is why the files are 75 KB. Extract to a shared `optimization/data_loader.py` and import. Should cut each file by ~30%.
- `ARTIFACT_REPORT_PATH = Path(r"C:\Users\SIGMA\geminiantigravity\brain\059c9b32-.../wap_oos_report.md")` — a hardcoded absolute Windows path with someone's local GUID, written on every report update. Make it `os.environ.get("ARTIFACT_REPORT_PATH")` and no-op if unset.
- Report is titled `wap_oos_report.md` in the S1 file, and the combined section is headed "COMBINED 4-STRATEGY" while `strategies_to_run = ["S1_Liquidation"]` and the ensemble is 6 strategies. Stale copy-paste throughout.
- `SYMBOLS` disagrees across modules. Your prompt lists `MATIC, ATOM`; `opt_s1` lists `SUIUSD, TRXUSD` instead and omits both; `ensemble_master` uses only 8. Three sources of truth. Hoist to one `symbols.py`.
- `MONTHS` includes `("2026-06-09", "2026-07-09")` — today is 2026-07-31, so window 20 closed three weeks ago. Fine, but nothing validates that windows are in the past; a future-dated window would silently produce a zero-trade "FAIL".
- Bare `except Exception as e: print(...)` in `process_symbol` swallows everything and returns empty frames, so a genuine data-loading failure looks identical to "no trades". This is exactly how C1 stayed hidden. Log the traceback and re-raise unless `--soft-fail`.
- `warnings.filterwarnings('ignore')` at module top and again in `__main__` — hides the pandas `SettingWithCopyWarning` s that `df_comb[col] = df_alpha[col]` in `generate_features_standard` is almost certainly generating.
- `np.mean([d['roi_pct'] for d in monthly_details])` counts skipped windows as 0% ROI, understating the average. §3.2.
- `supervisor.py` opens/closes the log file per line in `read_stdout` — one `open()` + `write()` + `close()` syscall triple per engine output line. Hold the handle open and `flush()`.
- `supervisor.py::cleanup_chrome` runs `kill -f chrome / Stop-Process -Name chrome -Force` on every restart — that kills the operator's browser, not just the scraper's headless instances. Scope it to the child PID or a unique `--user-data-dir` marker.

View all changes

View all changes

6.5 Top 5, in order

- 1. **Fix** `trades.update(trades_vwap)` in `opt_s1::process_symbol` (§1.1). Nothing else matters until S1 produces trades. One line. Add the `n_gen == 0` assert so it can't regress.
- 2. **Kill the** `prob=1.0` $\rightarrow$ **1.5× Kelly fallback** (§1.3) and calibrate probabilities. You are currently taking maximum size in exactly the windows you understand least, and every reported ROI in a low-data window is inflated by up to 50%.
- 3. **Enjit the** `simulate_trade` **bar loop** (§6.2). 50-200× on the dominant cost, and it's what makes the adaptive-horizon search in §2.2 affordable.
- 4. **Adopt** `wf_criteria.py` : **75% pass rate**, `MIN_TRADES=8` $\rightarrow$ **UNDETERMINED, recency weighting** (§3.2). This is the direct answer to "window 1 poisons the run" — a 3-trade window becomes *absence of evidence*, not failure. It also collapses the `dd<15 / dd<6` contradiction. 
- 5. **Rebuild S1 on z-scored liquidation + volume confirmation + imbalance** (§4). The current entry is `pull_ema8 < -0.2`, which is S3's signal with a different name; S1 and S3 are near-duplicates, which is why "ensemble diversity" isn't buying you what you'd expect.

 View all changes

>

7. What I could not verify

Being explicit about the limits of this review, since I never got an interpreter:

- **No line numbers.** You asked for them; I cited function names and exact code quotes instead. My reads came through a markdown-rendering fetch that strips line numbering, and fabricating numbers would be worse than omitting them. With a working checkout I'd give you `git blame -accurate refs`.
- **No measured trade counts.** §1.4 is arithmetic from the code plus the `diagnose_window1.py` script to get the real numbers. The `pct_timeout_288` field is the one that decides whether §2.2 is needed at all.
- **Files I did not read:** `Engine_1.py` (191 KB — I'd need ~15 chunked fetches, and it's the live system so I didn't want to guess at it), `opt_s4/s5/s6` (I confirmed `s2/s3` share `simulate_trade` and the 288 constant; `s4-s6` are 85-88 KB and near-certainly the same template, including possibly the same `.update()` clobber — **please grep** `trades.update()` **across s4-s6 before anything else**), `mt5_broker.py`, `scripts/`, `ml_trend_pull/`, `alpha_squeezer_v17` . I also never found `run_all_parallel.py` in the tree — it's not at repo root or in `optimization/`; it may be untracked or renamed.
- **Ensemble results look implausible and I could not check them.** `ensemble_monthly_oos.json` reports `mean_monthly_return_pct: 891.5` with all 18 months passing and min 474%. At fixed 1% risk on \$5,000, a 876% month means `total_r = 876` — 169 trades averaging +5.2R each with a 71% win rate. Sustained across 18 windows and six years of regimes, that is not a plausible OOS result; it's the signature of either the same class of bug as C1, or `run_monthly_oos` sizing off `total_r` rather than compounding equity. Given C1, C2, and C3 all inflate results in this codebase, I'd treat `AUDIT_REPORT.md` 's "Status: PASS" as unverified until re-run against the patched simulator. I'd want to look at this before you push any ensemble params to the live engine.

Let me know how you want to handle the clone and I'll turn all of the above into actual commits on `arena/019fba0e-coinglass-trading`.

 Questions dismissed 



The questions were skipped, so I'll leave the decision open rather than assume one.

Summary

I could not do the hands-on part of this task. The sandbox never initialized — every `bash`, `write_file`, and `edit_file` call returned  `clone failed: Cloning into '/home/user/coinglass-trading'...` across ~50 retries spanning the session. The likely cause is `backtesting_data/` : 32 parquet files committed directly to git (ADA summary alone is 11 MB, BNB 13 MB), which puts the clone well past a few hundred MB.

So: **no checkout, no Python, no commits, no pushes.** I could not run the Window 1 diagnostic, could not execute any patch, and could not verify anything empirically.

What I did instead was read the repo over the GitHub API and raw endpoints — all 9 chunks of `opt_s1_colab_standalone.py`, the relevant parts of `opt_s2 / opt_s3`, plus `supervisor.py`, `data_paths.py`, `optimization/oos_simulator.py`, `oos_simulator_liquidation.py`, `ensemble_master.py`, `ensemble_monthly_oos.json`, and `AUDIT_REPORT.md`. The analysis above is grounded in code I actually read and quoted.

The headline finding contradicts the premise you gave me. Window 1 is not failing because of the 288-bar cooldown. In `opt_s1::process_symbol`, `trades.update(trades_vwap)` overwrites the real S1 trade list with `[]`, because the `simulate_trades_vwap` stub returns `{'S1_Liquidation': []}` — the wrong dict key, copy-pasted from a sibling file. **Every symbol, every (tp, trail) combo, every window gets zero S1 trades.** Window 1 is just where the diagnostic prints first. My back-of-envelope on the cooldown says a 1-ATR stop on 15m crypto should produce hundreds of trades per window, not 0-3 — so I'd strongly suggest running `diagnose_window1.py` and checking `pct_timeout_288` before touching any cooldown logic.

Three caveats on what I delivered:

- **No line numbers**, which you explicitly asked for. My reads came through a markdown-rendering fetch that strips them. I cited function names and exact code quotes instead rather than fabricate references.
- **The patches are untested.** They're diffs written against code I read, not code I ran. Treat them as reviewed proposals.
- **I never read** `Engine_1.py` (191 KB) or `opt_s4/s5/s6`. Those three are 85-88 KB and near-certainly share the same template as `s1` — **please grep -n 'trades.update('** `opt_s*.py` **before anything else.** I also could not find `run_all_parallel.py` anywhere in the tree.

One thing I'd flag for its own attention: `ensemble_monthly_oos.json` claims a 891% mean

 View all changes

>

 View all changes

>

One thing to nag for its own attention: `ensemble_monthly_003.json` claims a 33.1% mean monthly return with 18/18 windows passing. At fixed 1% risk that implies ~876R in a month. Given that three separate bugs in this codebase (the S1 clobber, the `prob=1.0` $\rightarrow$ 1.5x Kelly fallback, and the test-window threshold selection) all inflate results in the same direction, I would not push ensemble params to the live engine until those numbers are re-derived from a patched simulator.

To get me unblocked: the cleanest path is adding `backtesting_data/*.parquet` to `.gitignore` and moving the data to Git LFS or external storage. Otherwise I can keep handing you diffs in chat and you apply them locally.

 

 In the following powe...

 Attached in the pytho...

..
.
.
.
.
.

 singhksranbir0248@gmail.c...

[Terms of Use](#) [Privacy Policy](#) [Cookies](#)

What would you like to do?

 Add files

 

 coinglass-trading |  arena/019fba0e-coinglass-trading

No Changes

 Create PR